and comparing these snapshots, the pie chart doesn't smoothly transition from one state to another but instead distorts quite significantly.

The reason you see this wonky transition is because, as you learned earlier, the default data-binding key is array position. When the pie layout measures data, it also sorts it in order from largest to smallest, to create a more readable chart. But when you recall the layout, it re-sorts the dataset. The data objects are bound to different pieces in the pie chart, and when you transition between them graphically, you see the effect shown in figure 5.8. To prevent this from happening, we need to disable this sort:

```
pieChart.sort(null);
```

The result is a smooth graphical transition between `numTweets` and `numRetweets`, because the object position in the array remains unchanged, and so the transition in the drawn shapes is straightforward. But if you look closely, you'll notice that the circle deforms a bit because the default `transition()` behavior doesn't deal with arcs well. It's not transitioning the degrees in our arcs; instead, it's treating each arc as a geometric shape and transitioning from one to another.

This becomes obvious when you look at the transition from either of those versions of our pie chart to one that shows `numFavorites`, because some of the objects in our dataset have 0 values for that attribute, and one of them changes size dramatically. To clean this all up and make our pie chart transition properly, we need to change the code. Some of this you've already dealt with, like using key values for your created elements and using them in conjunction with exit and update behavior. But to make our pie pieces transition in a smooth graphical manner, we need to extend our transitions to include a custom tween to define how an arc can grow or shrink graphically into a different arc.

Listing 5.2 Updated binding and transitioning for pie layout

```
pieChart.value(function(d) {
                    return d.numRetweets;
                });

d3.selectAll("path").data(pieChart(nestedTweets.filter(function(d) {
                                    return d.numRetweets > 0;
                                })),
                                function (d) {
                                     return d.data.key;
                                }
)
.exit()
.remove();

d3.selectAll("path").data(pieChart(nestedTweets.filter(function(d) {
        return d.numRetweets > 0;
})),
function (d) {
         return d.data.key}
)
```

Updates the function that defines the value for which we're drawing arcs

Binds only the objects that have values, instead of the entire array

User id becomes our key value; this same key value needs to be used in the initial enter() behavior

Removes the elements that have no corresponding data